UNIVERSITY OF PISA

Artificial Intelligence and Data Engineering Master Program

Business and Project Management Project

BuildSmart AI

Integrated Construction Site Management System
with AI Assistant RAG

Group members:

Rosario Ruisi
Leonardo Cecchini
Matteo Protopapa

`https://github.com/Leo-Cecchini/Business-Project/tree/main`

ACADEMIC YEAR 2024/2025

Contents

Chapter 1

Introduction

This documentation describes the design and development of **BuildSmart AI**, an innovative software platform dedicated to process optimization in the construction sector (*Construction Management*). Developed as part of the *Business and Project Management* course, the project aims to bridge the technological gap that often affects small and medium Enterprises in the construction industry, offering a tool that combines management robustness with the flexibility of Artificial Intelligence. The following sections will analyze the reference scenario, the motivations behind the development, and the strategic objectives of the proposed solution.

1.1 Context and Motivation

With the progress of Generative AI (GenAI), the number of sectors involved in the technological wave triggered by the rapid growth of artificial intelligence has increased exponentially. This has made it nearly impossible for executives across all industries to remain indifferent or resistant to the introduction of such technology in support of their operational processes, aiming not only to accelerate them but also to automate them.

We have chosen to focus our idea on the construction industry, which is characterized by high operational complexity involving the coordination of heterogeneous resources (masons, electricians, plumbers), strict adherence to deadlines, and the careful management of material costs subject to fluctuations. For small and medium enterprises, these activities are often managed through fragmented tools: Excel spreadsheets for costs, WhatsApp for communication, and disorganized local folders for technical drawings.

1.2 Project Objectives

Our project aims to resolve this fragmentation by providing a single unified web application that allows for the management of various construction sites, workers,

and materials, while leveraging GenAI to speed up tedious processes and save a significant amount of time. The main objectives are:

1. **Centralization:** A single source of truth for Construction Sites (Projects), Costs, and Workforce.
2. **Automation:** Automated scheduling checks to prevent overlaps (overbooking) and verify skills.
3. **Intelligent Assistance:** A conversational AI that acts as a "Site Co-pilot," capable of reading specific PDF contracts or technical specifications and answering questions such as "What is the concrete grade for the foundations?".

Chapter 2

System Architecture

The definition of the architecture for **BuildSmart AI** arises from the need to combine the stability required for a critical business management system with the flexibility demanded by modern Generative Artificial Intelligence models.

The system design was guided by the principles of **modularity** and **maintainability**, adopting a service-oriented approach that ensures decoupling between the business logic (construction site management, resources, costs) and the AI inference engine. This separation allows for updating language models or scaling the vector database without impacting the stability of daily operations. The following sections will analyze in detail the technological choices, the component diagram, and the data flows governing the RAG pipeline.

2.1 Technology Stack

The choice of technologies was driven by the necessity to create a system that is both responsive for the end user and flexible for the integration of Artificial Intelligence components. The system adopts a modern Full-Stack approach, which is fully containerized.

Layer	**Chosen Technology**
Frontend	**React** (Vite), Tailwind CSS, React Query
Backend	**Python Flask** (Blueprints Pattern)
Database	**MongoDB** (Data), **Qdrant** (Vector Store)
AI Model	**Google Gemini 2.5 Flash** (via LangChain)
Deployment	**Docker Compose**

Table 2.1: Technology Stack Overview

2.1.1 Frontend: Interactivity and Performance

For the user interface, the **React** library was chosen, served via the **Vite** build tool. This combination ensures fast loading times and a fluid user experience (Single Page Application).

- **State Management:** The use of `React Query` (@tanstack/react-query) allows for managing client-side data caching, reducing redundant server calls and keeping the dashboard synchronized with the database in real-time.
- **Styling:** For styling, **Tailwind CSS** was adopted, a utility-first framework that enabled the rapid development of a responsive and consistent design without the overhead of traditional CSS files.

2.1.2 Backend: Modularity and AI Logic

The server-side is developed in **Python 3.11**, the language of choice for AI applications. **Flask** was selected as the web framework for its lightness and flexibility.

- **Blueprints Architecture:** The backend is not monolithic but divided into logical modules (Blueprints) separated by functionality (e.g., `api/projects`, `api/workers`, `api/chat`). This facilitates maintenance and isolated testing of components.
- **AI Integration:** Python allows for the native import of `LangChain` and `SentenceTransformers` libraries, which are essential for orchestrating the RAG flow and communication with the Gemini model.

2.1.3 Persistence Layer: Hybrid Data

The nature of construction data, which is often heterogeneous and hierarchical, required a dual persistence strategy:

1. **MongoDB (Operational Database):** A NoSQL database was preferred for its ability to handle flexible schemas. Entities such as "WorkItems" are saved as embedded documents within Projects, optimizing read performance and avoiding complex JOIN operations typical of relational databases. Interaction occurs via the `MongoEngine` ODM.
2. **Qdrant (Vector Store):** For semantic search, **Qdrant** was implemented—an open-source vector database running locally in a container. It stores the *embeddings* (mathematical representations) of uploaded documents, allowing the chatbot to retrieve information based on concept similarity rather than just keywords.

2.1.4 AI Engine: LLM and Orchestration

The "brain" of the system consists of **Google Gemini** models (Flash version), chosen for the excellent balance between inference speed and cost-effectiveness. The orchestration of calls and prompt management is handled by the **LangChain** framework, which manages conversation memory and the insertion of context retrieved from documents.

2.2 Architectural Diagram

The diagram shown in Figure 2.1 illustrates the high-level architecture of **BuildSmart AI**. The system is structured according to a logical microservices pattern, where each component has a unique and well-defined responsibility.

The main communication flow is orchestrated by the **Flask Backend**, which acts as middleware between the user interface, persistent data, and artificial intelligence services.

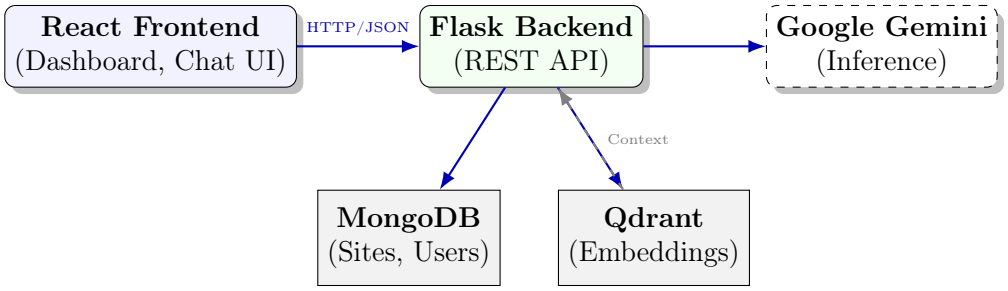

Figure 2.1: High-Level System Architecture

2.2.1 Data Flow Analysis

As highlighted in the diagram, interaction between components follows three main directions:

1. **Operational Flow (Frontend ↔ Backend ↔ MongoDB):** Standard user requests (site creation, worker assignment) travel via HTTP protocol transmitting JSON payloads. The Backend validates these requests and updates the persistent state on **MongoDB**, which serves as the "Single Source of Truth" for structured data.

2. **Semantic Flow (Backend ↔ Qdrant):** When technical documents are uploaded, the Backend processes them and saves their vector representations in **Qdrant**. The return arrow ("Context") indicates the retrieval of relevant text snippets during a search, which are used to enrich the AI prompt.

3. **Inference Flow (Backend ↔ Gemini):** The dashed **Google Gemini** node represents an external (cloud) service. The Backend sends the enriched prompt (user question + Qdrant context + MongoDB data) to Google APIs and receives a natural language response, which is then forwarded to the Frontend.

2.3 The RAG Pipeline (Retrieval-Augmented Generation)

The most advanced module of BuildSmart AI is the RAG system, designed to overcome the knowledge limits of general-purpose LLM models. Since a model like Gemini cannot know the private contents of a contract uploaded seconds prior, the system dynamically injects this knowledge at query time.

The pipeline's operation is divided into two distinct phases: **Indexing** (asynchronous write process) and **Inference** (synchronous read process).

2.3.1 Phase 1: Ingestion and Indexing

When a user uploads a technical document (PDF, Specifications, Regulations) to the project dashboard, the Backend activates the following processing chain:

1. **Extraction and Cleaning:** The file is read via specific parsers (e.g., `PyPDFLoader`) to extract raw text, removing redundant headers or unnecessary formatting characters.

2. **Chunking:** The continuous text is divided into logical segments (*chunks*) of fixed sizes. We configured a splitter with a **1000-character** window and a **200-character overlap**. Overlap is crucial for maintaining context between sentences that might be cut in half between two blocks.

3. **Vector Embedding:** Each chunk passes through a local embedding model (based on `SentenceTransformers/all-MiniLM-L6-v2`). This model converts text into a dense 384-dimensional numerical vector representing its semantic meaning.

4. **Storage in Qdrant:** Vectors are saved in the Qdrant database. Each vector is associated with a metadata *payload* containing the `project_id`. This step ensures **data segregation**: documents from "Project A" will never be retrieved to answer questions about "Project B."

2.3.2 Phase 2: Retrieval and Generation

When the user asks a question in the chat, the reverse process occurs:

1. **Intent Analysis and Routing:** Before querying the vector database, the system analyzes the question to understand if the user is looking for documents, materials, staff, a cost estimate, or web information.

2. **Query Embedding:** The user's question (e.g., "What is the concrete grade?") is converted into a vector using the same model from the ingestion phase.

3. **Similarity Search:** Qdrant calculates the "cosine distance" between the query vector and millions of saved vectors, applying a strict filter: `filter: { project_id: current_id }`. The 3-5 most similar (relevant) text fragments are returned.

4. **Context Injection:** The system builds a prompt for the LLM following this template:

   "You are a construction assistant. Use ONLY the following context to answer the question. Context: [Fragment 1]... [Fragment 2]... Question: ..."

5. **Generation:** Google Gemini processes the prompt and generates a natural response, implicitly citing the information contained in the retrieved fragments.

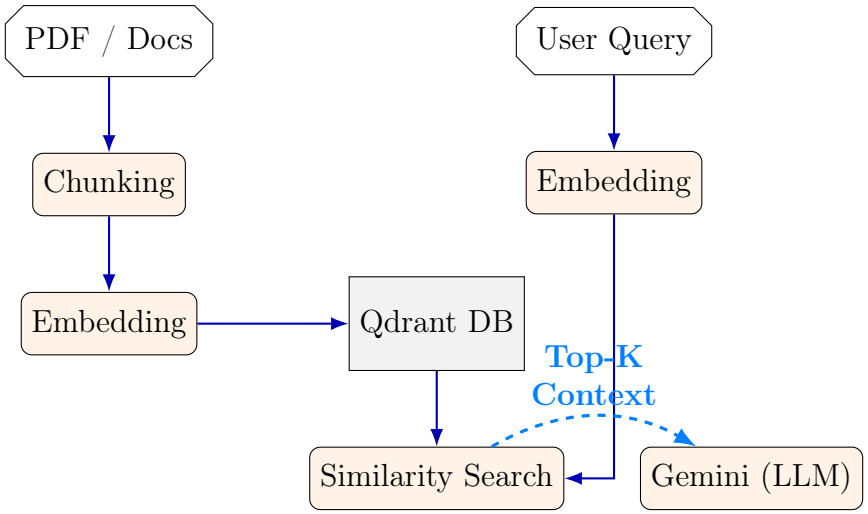

Figure 2.2: Logical RAG Pipeline Flow: Ingestion vs Retrieval

2.4 Intent Analysis (Intent Routing)

Intent analysis is the system component used to route the question to the most appropriate execution engine, avoiding unnecessary queries to the vector database (RAG) when the answer resides in a structured database or requires an external search.

Router Mechanics
The system uses a hybrid approach, combining Heuristics (Regex/Keywords) for speed and LLM (Large Language Model) for complex semantic understanding.

2.4.1 Heuristic Recognition (Fast Path)

For common operational queries, the system uses the `utils/intent_router.py` file to identify specific patterns without invoking the LLM, reducing latency and costs:

- **Materials:** Identifies terms like "SKU," "supply," "concrete," or units of measurement (kg, m2).
- **Documents:** Recognizes requests for analysis or comparison of PDF/Excel files (e.g., "compare the quote...").
- **Estimation:** Identifies technical parameters (sqm, thickness, lighting points) to activate the cost estimation module.
- **Web search:** Intercepts queries about weather, regulations (UNI, ISO, 81/08), or market prices to trigger an online search.

2.4.2 LLM Recognition (Deep Path)

For Staff management, the system delegates analysis to Gemini with a structured JSON output. This allows for managing:

- **Operations:** Counts, lists, roles, or clarifications.
- **Filters:** Availability (e.g., "who is free?"), hour limits, and specific roles (e.g., "plumber").
- **Context Awareness:** Ability to reuse previous context (e.g., "and of these, who is in Rome?").

2.4.3 MongoDB Integration (Quick DB Answer)

A critical part of the analysis occurs in the backend (`routes/chat.py`), where the system attempts a fast/direct response from the DB. If the question concerns data recorded in MongoDB (e.g., "How many total workers do we have?" or "Who is assigned to Project X?"), the router directly queries the `workers` or `projects` collections.

2.5 Memory and Context Management

Intent analysis does not happen in isolation for each message but is embedded in a conversation flow that tracks previous intentions. This process is essential for making the interaction natural.

2.5.1 The Role of `last_context`

In the `utils/intent_router.py` file, the `route()` function accepts an optional parameter called `last_context`. This object contains the intent extracted from the previous message.

When the user sends a new question, the system:

- Retrieves the intent of the last exchange from the session.
- Passes it to the LLM along with the new question.
- The LLM decides if the new request is a "Clarify" or if it should set the `reuse_last:true` flag.

2.5.2 Resolving Follow-ups

The system manages memory through two main mechanisms:

- **Filter Refinement**: If the user asks "Show me the bricklayers" (Intent: STAFF, Role: mason) and then adds "Only those who are free," the router recognizes that the "role" should be maintained from the previous context, simply adding the `free_only:true` filter.
- **Co-reference**: If the user asks for information about a specific site and then asks "How many workers are there?", the router maintains the project_id or the site name as the pivot for the search even for subsequent queries.

2.5.3 Session Persistence (Backend)

In the `routes/chat.py` file, memory is ensured by integration with Flask sessions and a unique identifier (session_id or sid):

- Every `/api/chat` request sends the session_id.
- The ChatService retrieves the history from the database (MongoDB) to provide the LLM with previous messages.
- This allows for injecting into the prompt not only the documents retrieved from Qdrant but also the message history, enabling Gemini to resolve pronouns (e.g., "How many are needed?": "many" refers to the cement bags mentioned earlier).

2.5.4 Context Clearing

To avoid "hallucinations" or conflicts between different topics, the intent context is reset if:

- A drastic change in topic is detected (e.g., switching from staff search to web regulation search).
- The user starts a new chat session.

Chapter 3

Data Model Implementation

Data management in the construction industry presents unique challenges: the structure of a construction site can vary enormously from one project to another, and price lists change based on geographical regions. To address this variability, the persistence layer of **BuildSmart AI** was built on **MongoDB**, a document-oriented NoSQL database.

However, to ensure the data integrity required for a corporate application, database access is mediated by **MongoEngine**, an ODM (Object-Document Mapper) for Python. This hybrid approach allows us to define rigorous schemas ("Schema Enforcement") while maintaining the flexibility to add custom fields or metadata without having to perform costly database migrations.

The following sections analyze the main entities (Documents) that constitute the core of the system.

3.1 Project Entity (ProjectDoc)

The ‘ProjectDoc‘ document represents the root entity of the system. A critical architectural choice was the adoption of the **Embedding** pattern for managing work tasks (‘WorkItems‘).

Instead of saving individual tasks in a separate collection and linking them via JOINs (as would happen in SQL), the tasks are saved directly within the project document.

- **Advantage:** When the user opens the site dashboard, the backend performs a single read operation (O(1)) to retrieve the project and all its tasks, ensuring near-instant loading times.
- **Structure:** The model includes fields for geolocation (useful for retrieving regional prices) and a list of embedded work items.

```
class ProjectDoc(Document):
    """
    Represents a construction site. Includes metadata and task
    list.
    """
    name = StringField(required=True)
    status = StringField(choices=['planning', 'active', '
    completed'])
    project_date = DateField(default=datetime.utcnow)

    # Structured Address (Embedded)
    addresses = EmbeddedDocumentField(AddressDoc)

    # List of specific work tasks for this site
    # Embedding avoids multiple queries to populate the
    dashboard
    works = ListField(EmbeddedDocumentField(WorkItem))

    # Reference to vector documents (for RAG)
    meta = {'collection': 'projects'}

class WorkItem(EmbeddedDocument):
    """
    A specific task (e.g., 'Floor Installation')
    """
    work_name = StringField(required=True)
    status = StringField(default='planned')

    # Temporal Planning
    start_date_planned = DateField()
    end_date_planned = DateField()
    duration_estimated_hours = FloatField()

    # Worker Assignment (List of IDs referring to the Workers
    collection)
    workers = ListField(StringField())
```

Listing 3.1: ProjectDoc and WorkItem Model Definition

3.2 Worker Entity (WorkerDoc)

The ‘WorkerDoc‘ collection manages the workforce records. In addition to standard data, this model has been enriched to support the **Smart Scheduling** algorithm:

- **Skills:** A list of strings (e.g., `["scaffolding", "crane_license"]`) that the AI uses to verify if a worker is qualified for a task.
- **Home City:** Used to calculate geographical availability and travel costs.

```
class WorkerDoc(Document):
    name = StringField(required=True)
    role = StringField(choices=['bricklayers', 'electrician',
    'plumber', 'driver'])
    is_active = BooleanField(default=True)
    hourly_rate = FloatField()

    # Data for the allocation algorithm
    home_city = StringField()
    home_region = StringField()
    skills = ListField(StringField())

    meta = {'collection': 'workers'}
```

Listing 3.2: WorkerDoc Model with Skills

3.3 Cost and Price List Management (Bill of Quantities & Pricelists)

A distinctive feature of BuildSmart AI is the dynamic management of costs, designed to solve the problem of price variability in the construction sector. The system decouples the concept of "Material" (technical data) from its "Price" (economic value linked to the territory), allowing for precise estimates without data duplication.

3.3.1 PricelistDoc (Regional Price Lists)

The ‘PricelistDoc‘ collection acts as a geolocated economic register. Each document in this collection defines costs for a specific geographical area and is divided into two main dictionaries:

- **Materials:** Maps the material SKU code to its local unit price (e.g., a bag of cement might cost €5.00 in Milan and €4.50 in Naples).
- **Wages:** Defines the minimum hourly rates for labor roles (e.g., the hourly cost of a specialized electrician in the Lazio region).

Hierarchical Lookup Algorithm

To determine the correct price to apply to a project, the estimation service (‘EstimateService‘) implements a **hierarchical fallback** logic. When a resource price is requested for a project located in a given city, the system performs the following checks in order:

1. **City Match:** It searches for a specific price list for the project’s city (e.g., ‘city="Milan"‘).

2. **Regional Match:** If not found, it searches for a price list for the corresponding region (e.g., ‘region="Lombardy"‘).

3. **National Fallback:** If both are missing, it uses the base ("Default") price list to ensure a valid estimate is still provided.

```
class PricelistDoc(Document):
    """
    Geolocated price list.
    """
    region = StringField(required=True)  # e.g., "Tuscany"
    city = StringField()                 # e.g., "Pisa" (
   Optional)

    # Material Code -> Price (   ) Dictionary
    # Example: { "CEM-001": 5.50, "MAT-BLK-20": 0.90 }
    materials = DictField()

    # Role -> Hourly Rate (   /h) Dictionary
    # Example: { "mason": 25.00, "electrician": 32.00 }
    wages = DictField()

    meta = {
        'collection': 'pricelists',
        'indexes': [
            {'fields': ('region', 'city'), 'unique': True}
        ]
    }
```

Listing 3.3: PricelistDoc Model Structure

3.3.2 ComputoDoc (Bill of Quantities and Cost Estimation)

The ‘ComputoDoc‘ represents the final accounting document associated with a project (Bill of Quantities). Unlike a simple static quote, it is structured to be generated and updated via AI.

It contains a list of ‘ComputoItem‘ entries, each including a description, unit of measure, quantity, and calculated prices. The ‘grand_total‘ field is automatically recalculated by the backend whenever an entry is modified, ensuring accounting consistency.

```
class ComputoItem(EmbeddedDocument):
    """
    Single line of the Bill of Quantities.
    """
    code = StringField()        # Reference code (optional)
    description = StringField() # Full description of the work
```

```
7
8       unit_of_measure = StringField() # sqm, cum, kg, pcs
9       quantity = FloatField()
10
11      unit_price = FloatField()    # Unit price (derived from
    Pricelist)
12      total_price = FloatField()  # quantity * unit_price
```

Listing 3.4: ComputoItem Definition

3.3.3 ComputoDoc Structure

The ‘ComputoDoc‘ stores the Estimated Bill of Quantities. This document is often automatically generated by the AI starting from an uploaded PDF or a textual description.

```
1 class ComputoDoc(Document):
2     project_id = ObjectIdField(required=True)
3
4     # List of cost items
5     bill_of_quantities = ListField(EmbeddedDocumentField(
    ComputoItem))
6
7     # Automatically calculated totals
8     grand_total = FloatField()
9     created_at = DateTimeField()
10
11 class ComputoItem(EmbeddedDocument):
12     description = StringField()
13     unit_of_measure = StringField() # e.g., sqm, cum, kg
14     quantity = FloatField()
15     unit_price = FloatField()
16     total_price = FloatField() # qty * unit_price
```

Listing 3.5: Bill of Quantities Structure

Chapter 4

Key Features

BuildSmart AI has been designed to cover the entire lifecycle of a construction contract, from initial cost estimation to daily site management. The system's functionalities can be grouped into three fundamental pillars:

1. **Operational Management:** Tools for project creation, defining work phases (WBS - Work Breakdown Structure), and assigning human resources.
2. **Financial Control:** A module dedicated to the drafting and management of the Estimated Bill of Quantities, enhanced by automated data extraction algorithms.
3. **Decision Support:** An intelligent conversational interface that reduces information search times and supports the manager in critical decision-making.

The following sections analyze the implementation logic of these modules in detail.

4.1 Smart Scheduling

Workforce management is one of the most critical and error-prone activities. The system implements a proactive validation engine, the `ScheduleService`, which intervenes whenever a manager attempts to assign a worker to a task (`WorkItem`).

The **Capacity Check** process performs three sequential checks to ensure scheduling consistency:

1. **Contractual Status Verification:** The system primarily checks if the worker is marked as `is_active` in the database. Workers on sick leave, vacation, or with terminated contracts are automatically excluded.
2. **Temporal Conflict Detection (Overlap):** The system cross-queries all active projects ('ProjectDoc') to verify if the worker's ID is already committed to another task that overlaps, even partially, with the selected date range.

$$\text{Conflict} \iff \exists \text{ task } t : (t_{\text{start}} < \text{new}_{\text{end}}) \land (t_{\text{end}} > \text{new}_{\text{start}})$$

3. **Skill Compatibility (Skill Match):** (Optional) The system compares the worker's skill tags (e.g., "electrician") with the type of work required. If an inconsistency is detected (e.g., a mason assigned to electrical panel wiring), a non-blocking *warning* is generated for the manager.

4.2 AI-Assisted Bill of Quantities

The cost management module modernizes the traditional workflow based on static spreadsheets. The key feature is **AI-Driven Data Extraction**, which allows for the rapid digitization of paper quotes or PDFs.

The operational flow is as follows:

- **Ingestion:** The user uploads a PDF file (e.g., a tender specification).
- **LLM Processing:** The backend sends the text content to Google Gemini with a system prompt that enforces a strict JSON output structure. The AI is instructed to identify: *Item Description*, *Unit of Measure*, *Quantity*, and *Unit Price*.
- **Persistence:** The structured data is saved as a 'ComputoDoc'. From this point forward, the system automatically calculates partial totals and the 'grand_total', updating them dynamically if the user manually modifies a quantity.

4.3 Hybrid AI Chatbot (Intent Router)

The BuildSmart AI virtual assistant stands out from generic chatbots due to its "Hybrid" architecture. The system does not merely generate text; it acts as an intelligent router that classifies the user's intent before formulating a response.

This classification occurs in two primary modes:

- **DB-First Intent (Deterministic):** When the user asks operational questions about factual data (e.g., *"Who is working at the 'Rossi' site today?"* or *"What is the remaining budget?"*), the system recognizes the request and executes a direct query on MongoDB. **Advantage:** This approach eliminates the risk of AI "hallucinations", ensuring 100% accurate answers based on real data.
- **RAG Intent (Generative):** When the question concerns qualitative or technical information contained in documents (e.g., *"What are the specifications for acoustic insulation?"*), the system activates the RAG pipeline. It queries the Qdrant vector database, retrieves relevant fragments, and allows the LLM to generate a natural summary.

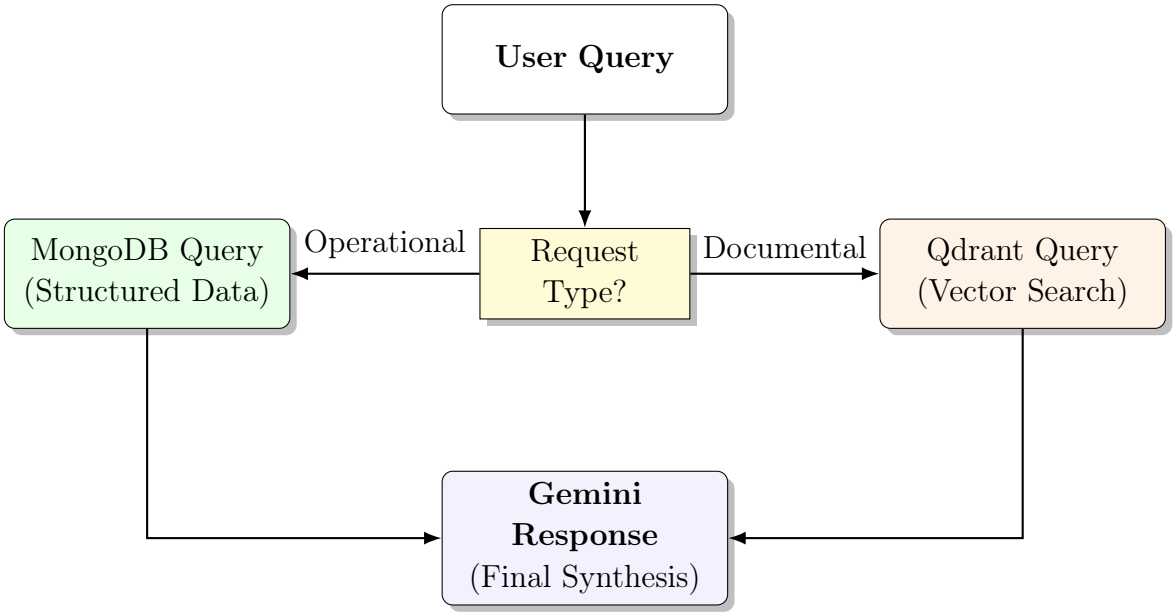

Figure 4.1: Logical Flow of the AI Intent Router

Chapter 5

User Interface

The interface of BuildSmart AI has been designed following *Clean Design* principles to ensure accessibility even for users who are not technically advanced, typical of construction small and medium enterprises. The primary goal of the UX (User Experience) is to reduce the "cognitive load": critical information (delays, costs, safety) must be visible at a glance, while advanced features (such as RAG) are accessible on-demand via the sidebar.

The frontend, developed in **React**, is fully responsive and organized into three macro-functional areas.

5.1 Dashboard and Project Overview

The *Dashboard* serves as the system's Command Center. Upon logging in, the manager sees a grid of cards representing active construction sites. Each card provides an immediate summary of the project's health:

- **Status Indicator:** A color-coded badge (Green/Yellow/Red) indicating whether the site is on schedule or delayed.
- **Progress Bar:** Displays the completion percentage based on closed *WorkItems* relative to the total.
- **Quick Actions:** Fast-access buttons to reach the bill of quantities or the chat without opening the full details.

5.2 Site Details and Planning

Clicking on a project opens the detailed view, which is divided into tabs. The **"Works"** tab is the operational heart of the system. Here, the interface changes paradigm, shifting to a task-oriented visualization.

- **Interactive List:** Tasks are listed chronologically. Each row shows planned vs. actual dates.

- **Site Manager:** On the right side there are 3 voices for managing the site:
  - **"Overview"**: Shows the principal information of the site with the main management features, related documents and the dedicated chat.
  - **"Computo Metrico"**: Shows the list of the voices in the computo metrico with all the details.
  - **"Work sequence"**: Shows the list of all the works to be done with deadlines, estimated hours and needed workers.

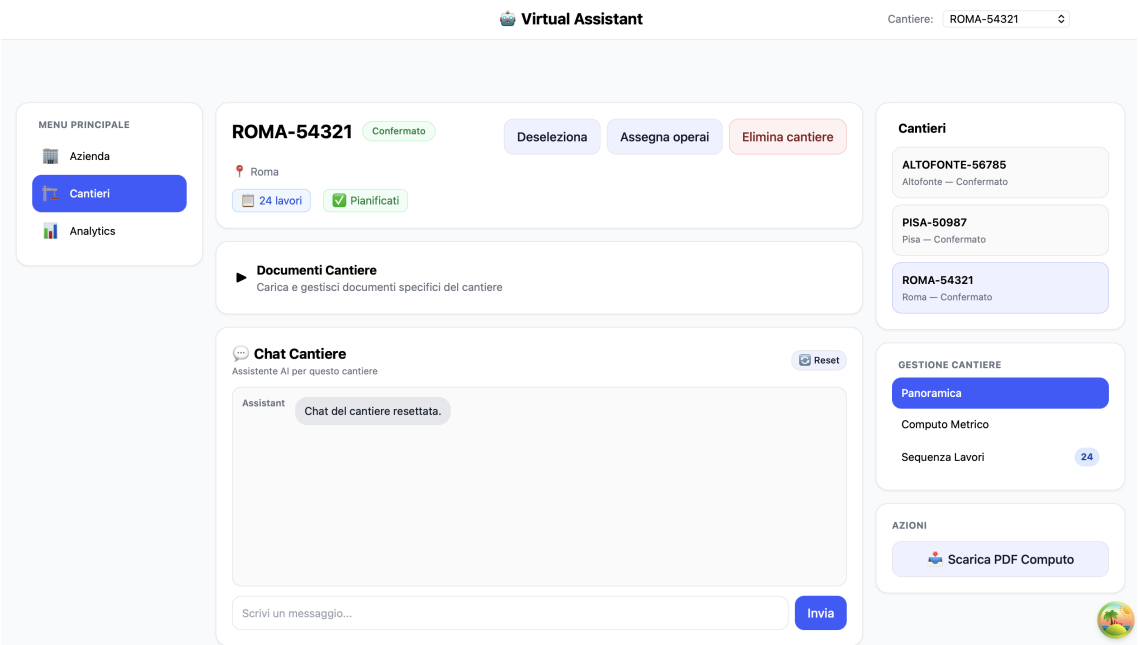

Figure 5.1: Site dashboard: main view

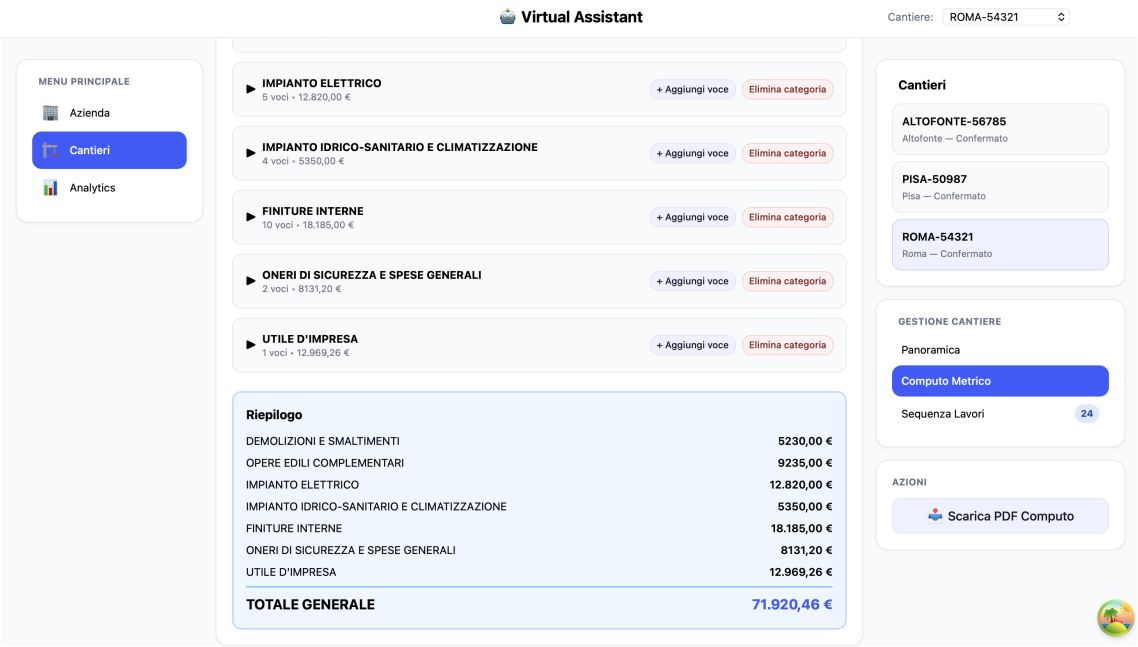

Figure 5.2: Site dashboard: computo metrico view

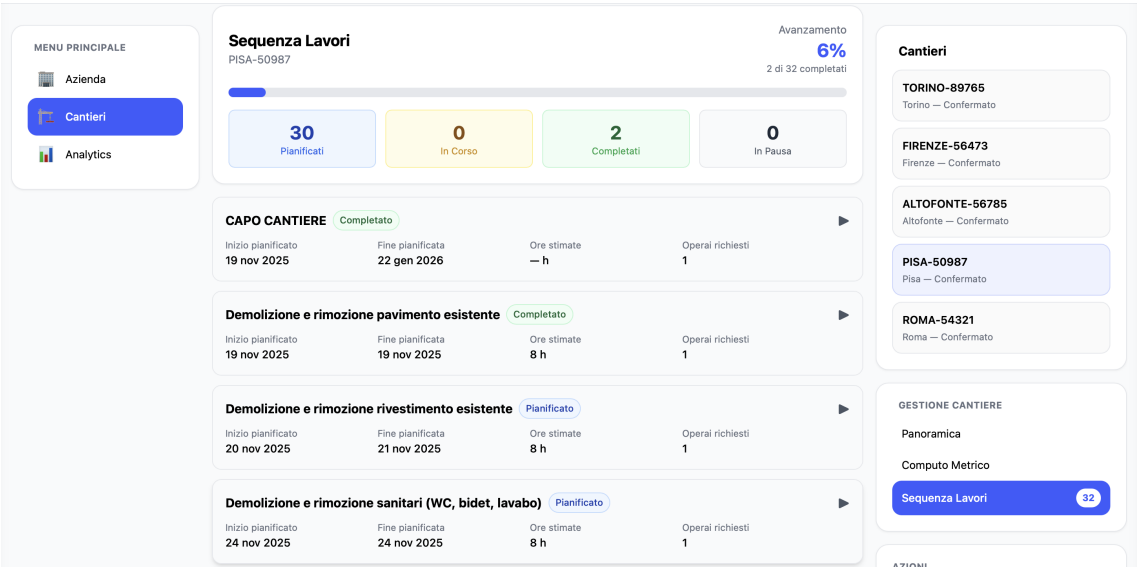

Figure 5.3: Site dashboard: work sequence view

5.3 SiteChat: The Contextual Assistant

To prevent the user from having to constantly switch pages to search for information, the **SiteChat** is implemented as a *Persistent Sidebar* on the right side of the screen.

UX Features of the Chat:

- **Context Awareness:** The chat "knows" which project the user is currently viewing. It is not necessary to specify "in the Via Roma site..."; the context is

implicit.

- **Streaming Response:** AI responses are generated in streaming (typewriter effect), providing immediate visual feedback to the user that the system is processing.
- **Citations:** When the system utilizes RAG, it can (in the advanced version) highlight which document it consulted.

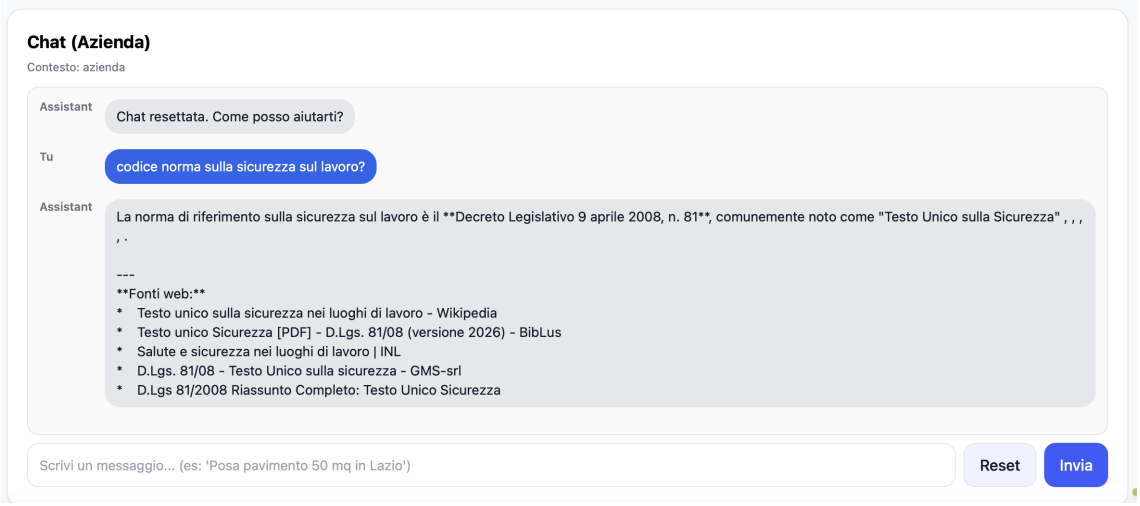

Figure 5.4: Main chat normative question

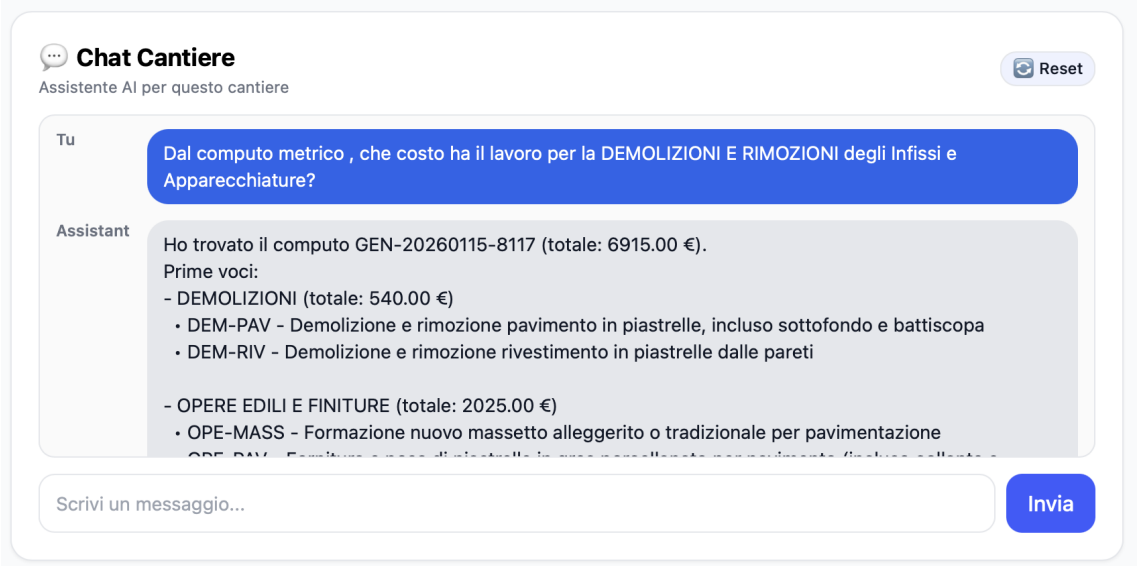

Figure 5.5: Site Chat Example with web search

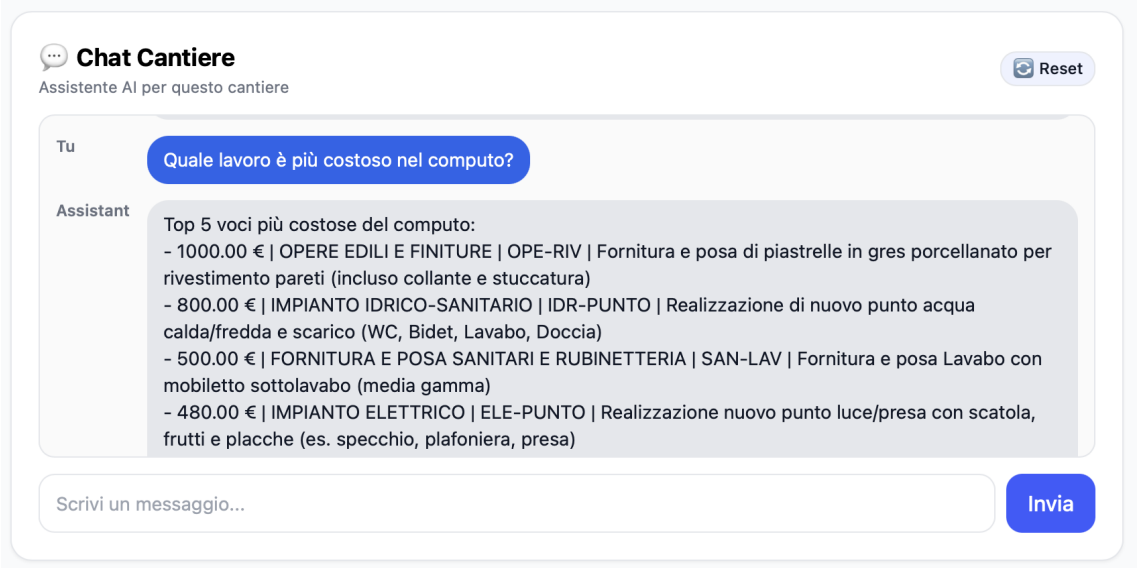

Figure 5.6: Site Chat Example with top 5 works

Chapter 6

Conclusions and Future Developments

The development of **BuildSmart AI** has demonstrated how the adoption of modern software architectures can radically transform operational management within the construction sector. The project has achieved its primary objective: overcoming the fragmentation of traditional tools (scattered spreadsheets, paper archives) through a centralized and intelligent platform.

6.1 Summary of Results

The analysis of the developed prototype reveals two main strengths:

- **Effectiveness of the Hybrid Approach:** The decision to combine a NoSQL database for structured data (MongoDB) with a vector engine for documents (Qdrant) has proven successful. The "Intent Router" allowed the system to get the best of both worlds: absolute precision for operational queries (e.g., shifts, costs) and the semantic flexibility of the LLM for technical consultation.
- **Architectural Scalability:** The use of Docker and the clear separation between the Frontend (React) and Backend (Flask) ensure that the system can evolve. Adding new "modules" (e.g., a billing system) would not require rewriting existing code, but only integrating new API endpoints.

In terms of operational impact, preliminary tests suggest that using the RAG pipeline for consulting specifications drastically reduces the time spent searching for technical information, allowing Site Managers to focus on the physical coordination of the site rather than bureaucracy.

6.2 Current Implementation Limitations

It is important to emphasize that the current version of BuildSmart AI represents a *Proof of Concept* (PoC). As such, it possesses intrinsic operational limitations related to both the RAG technology and the underlying Language Model used.

6.2.1 AI Model Dependency

The project relies on an external AI model (Google Gemini). Consequently, the accuracy of responses and results—especially regarding complex tasks like the automatic generation of the *Estimated Bill of Quantities*—is strictly correlated to the model's performance. For this prototype, the highest-performing model available was not utilized due to budgetary constraints and technical limitations (token limits and rate-limiting on free tiers). However, as a B2B product, in a production scenario, companies could invest in Enterprise subscription plans to utilize superior models, ensuring maximum precision. Furthermore, the rapid evolution of the AI sector suggests that the performance of "average" models will naturally increase over time.

6.2.2 RAG Operational Limits

The system has been optimized to answer construction-domain questions based strictly on the provided documentation. This entails:

- **Ambiguity Handling:** The chatbot may struggle to correctly interpret queries that are phrased too colloquially, ambiguously, or lack sufficient context.
- **Domain Coverage:** Responses are bound by the quality and completeness of the uploaded PDFs. Questions regarding topics not explicitly detailed in the documents (or general knowledge queries) may receive generic or "fallback" responses (e.g., "I cannot find this information in the documents").
- **Robustness:** Advanced *Guardrails* mechanisms to filter malicious or entirely off-topic inputs have not yet been implemented; a commercial release would require these additional security filters.

These limitations do not invalidate the proposed architecture but highlight the need for a subsequent phase of *Fine-Tuning* and large-scale testing prior to commercial deployment.

6.3 Future Developments (Roadmap)

Although the system is functional, several areas for improvement have been identified for future production deployment:

1. **Native Mobile Application (React Native):** Currently, the interface is optimized for desktop/tablet (office use). The next step involves developing a mobile app for workers, allowing them to:
   - View their assigned shifts.
   - Perform geolocated check-in/check-out at the construction site.
   - Upload photos of work progress directly into the project feed.
2. **"Hands-Free" Voice Interface:** On a construction site, using a keyboard is often inconvenient and unsafe. By integrating voice transcription APIs (such as OpenAI Whisper or native browser APIs), the *SiteChat* could be used vocally, allowing the site manager to query the AI while inspecting the works.
3. **Predictive Delay Analysis:** By leveraging the historical data stored in MongoDB, we intend to implement a classic Machine Learning module (e.g., Linear Regression or Random Forest) to estimate the probability of a task delay based on weather, complexity, and past performance of the assigned team.
4. **Intelligent Visual Monitoring (Computer Vision):** The integration of IoT cameras on-site, connected to computer vision models (e.g., YOLO), to automate monitoring:
   - **Safety Compliance:** Automatic detection of missing PPE (helmets, vests, harnesses) with immediate alerts sent to the safety manager.
   - **Automated Progress Tracking:** Visual analysis of the site status to automatically update the completion percentage of *WorkItems* (e.g., recognizing that a wall has been built), reducing manual data entry.
   - **Time & Attendance:** Automatic verification of entry and exit times for operational teams.
5. **Multi-Tenancy and SaaS:** Evolving the architecture to support multiple construction companies on the same instance (Software as a Service), implementing strict data segregation at the database level and a more granular Role-Based Access Control (RBAC) system.

Chapter 7

Demo and Presentation Scenario

This section illustrates the demonstration scenario designed to validate the core functionalities of our project. The demo simulates the typical workflow of a Site Manager, from site creation to querying the virtual assistant.

7.1 Demo Objective

The objective is to showcase the complete lifecycle of a construction project managed through the platform:

Project Creation → Bill of Quantities Generation (AI) → Smart Scheduling → Resource Allocation → Operational Assistance via Chat

7.2 Walkthrough End-to-End

7.2.1 Phase 1: Site Setup

1. Access the main dashboard (**Azienda**).
2. Click on "**Crea Cantiere**" and enter the essential metadata: Site Name, Start Date, Description.
3. Confirm creation and access the project detail view.

Expected Result: The system redirects to the specific site dashboard, showing empty tabs for *Computo*, *Lavori*, and *Chat*.

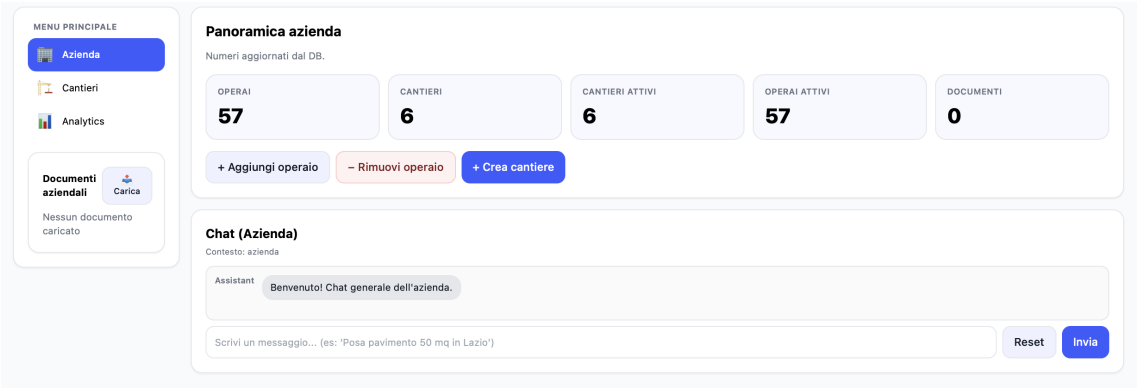

Figure 7.1: Main dashboard: create new site

7.2.2 Phase 2: Bill of Quantities Generation (AI)

1. Navigate to the **Computo Metrico** tab.
2. Upload the sample file (or paste the specifications text).
3. Click on **"Genera Computo"**.

Expected Result: After a few seconds, the system populates the table with a list of *Work Items*, including descriptions, estimated quantities, and units of measurement.

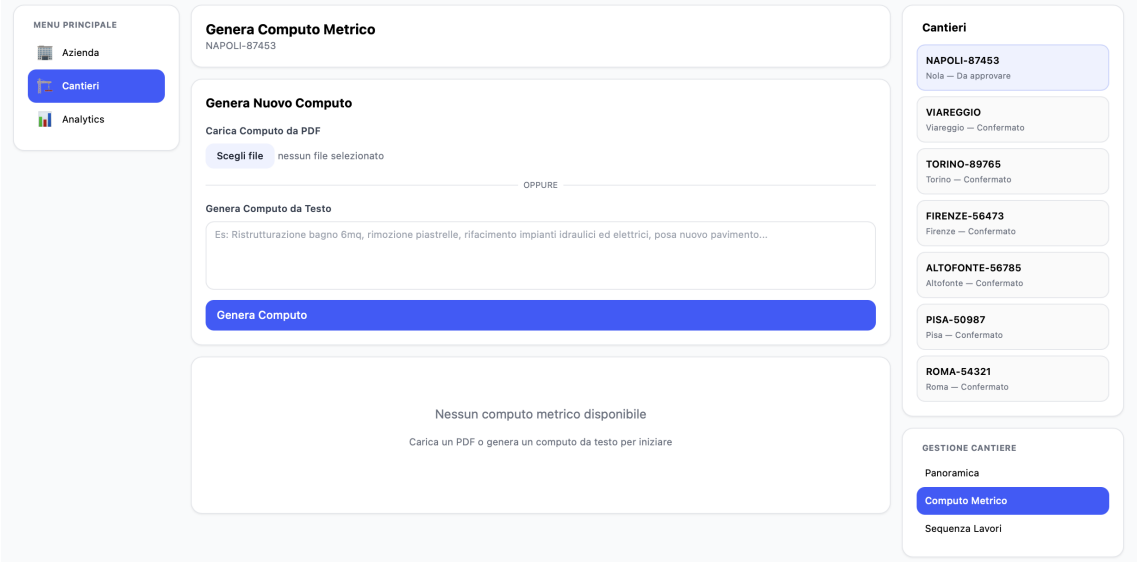

Figure 7.2: Bill of quantities: AI generation screen

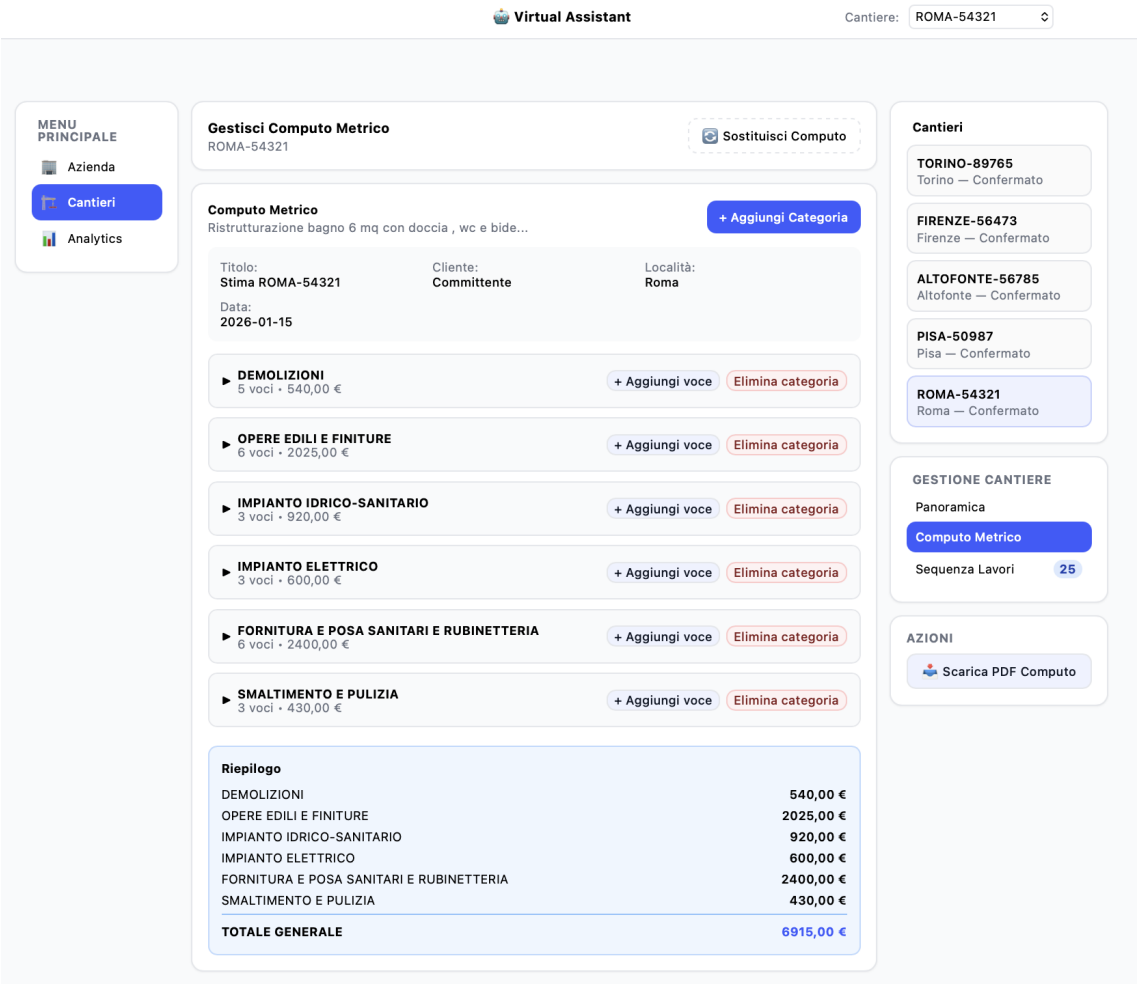

Figure 7.3: Bill of quantities: generated work items table

7.2.3 Phase 3: Planning (Smart Scheduling)

1. Switch to the **Sequenza Lavori** tab.
2. Select the start date and click the **"Genera Sequenza Lavori"** button.
3. The system automatically calculates for each computation item:
   - Logical start and end dates (sequential or parallel).
   - Estimated necessary workforce (*Crew Size*).

Expected Result: Visualization of a coherent timeline where each activity has a defined temporal duration.

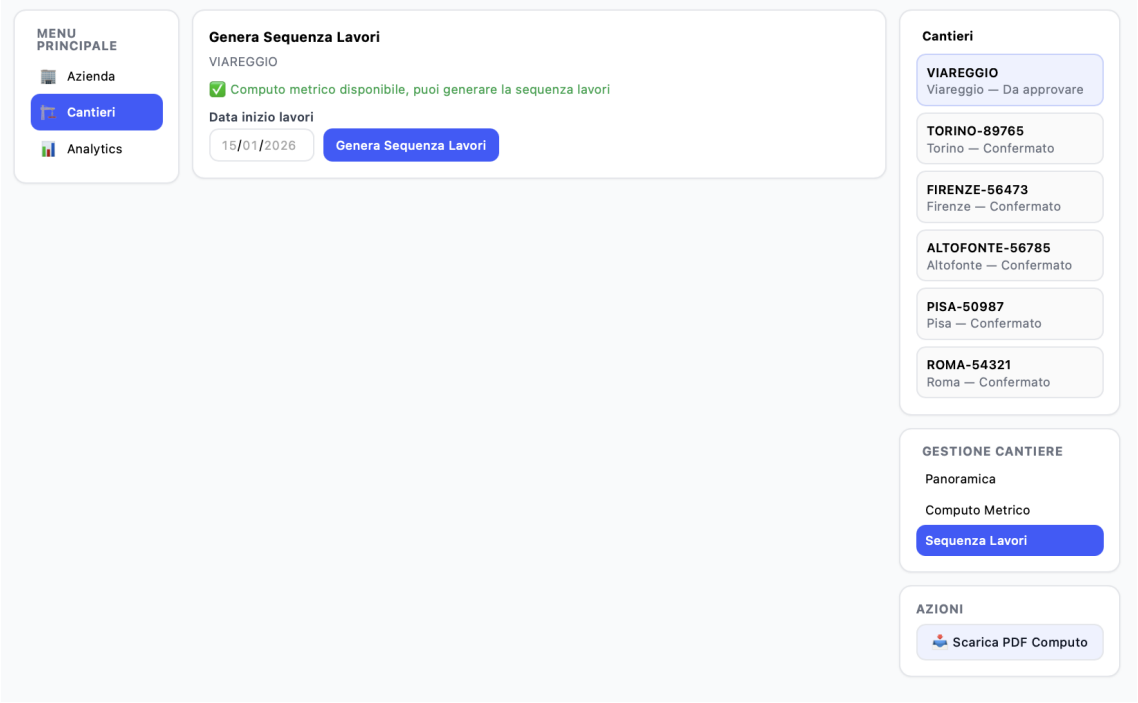

Figure 7.4: Work sequence: planning generation trigger

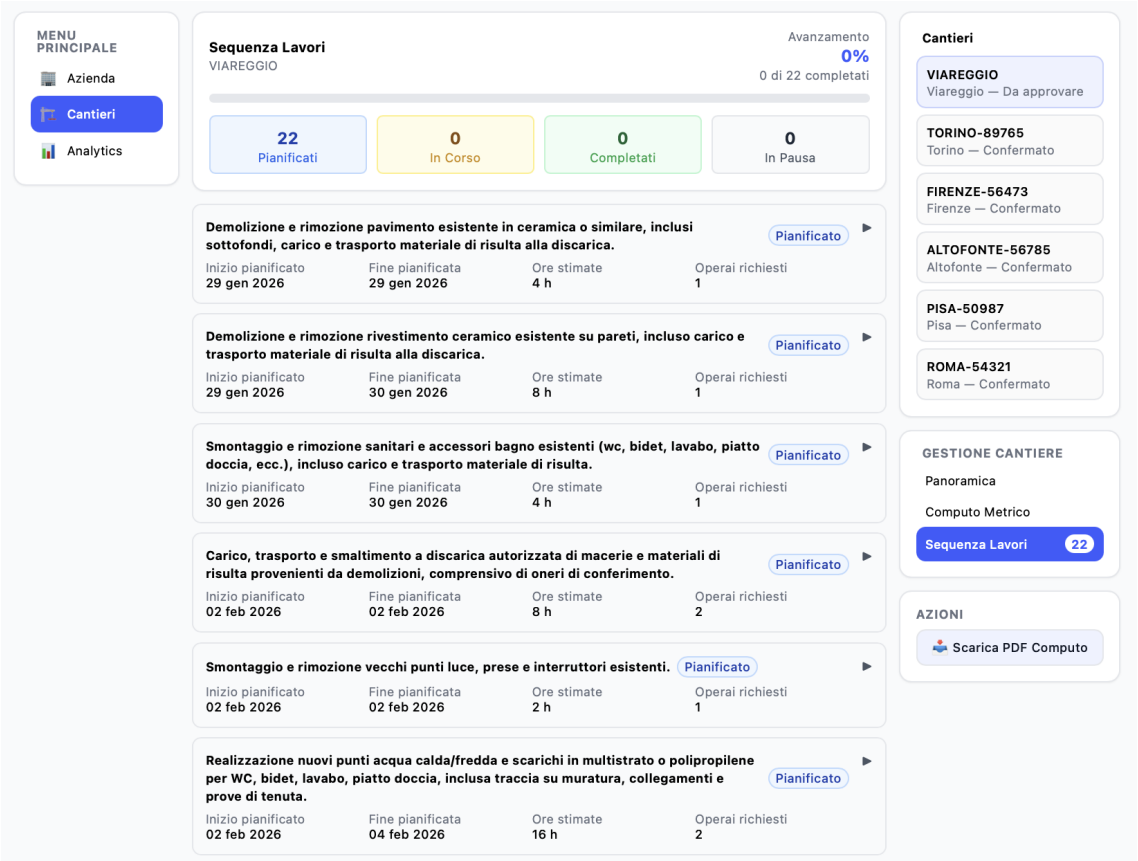

Figure 7.5: Work sequence: generated schedule timeline

7.2.4 Phase 4: Resource Allocation (Auto-Assign)

1. Click on **"Assegna Operai"**.
2. The system executes the *Capacity Check* algorithm:
   - Filters available workers for the indicated dates.
   - Matches skills (e.g., Electrician → Electrical System).
   - Mandatorily assigns a Site Manager (*Capocantiere*) to the project.

Expected Result: Each work card now shows photos or names of assigned workers. No overlap conflicts are generated.

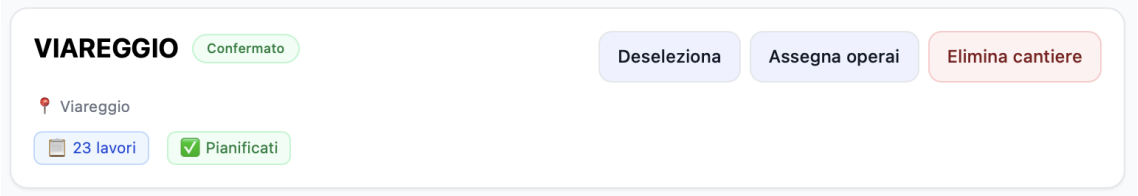

Figure 7.6: Resource allocation: "Assign workers" action

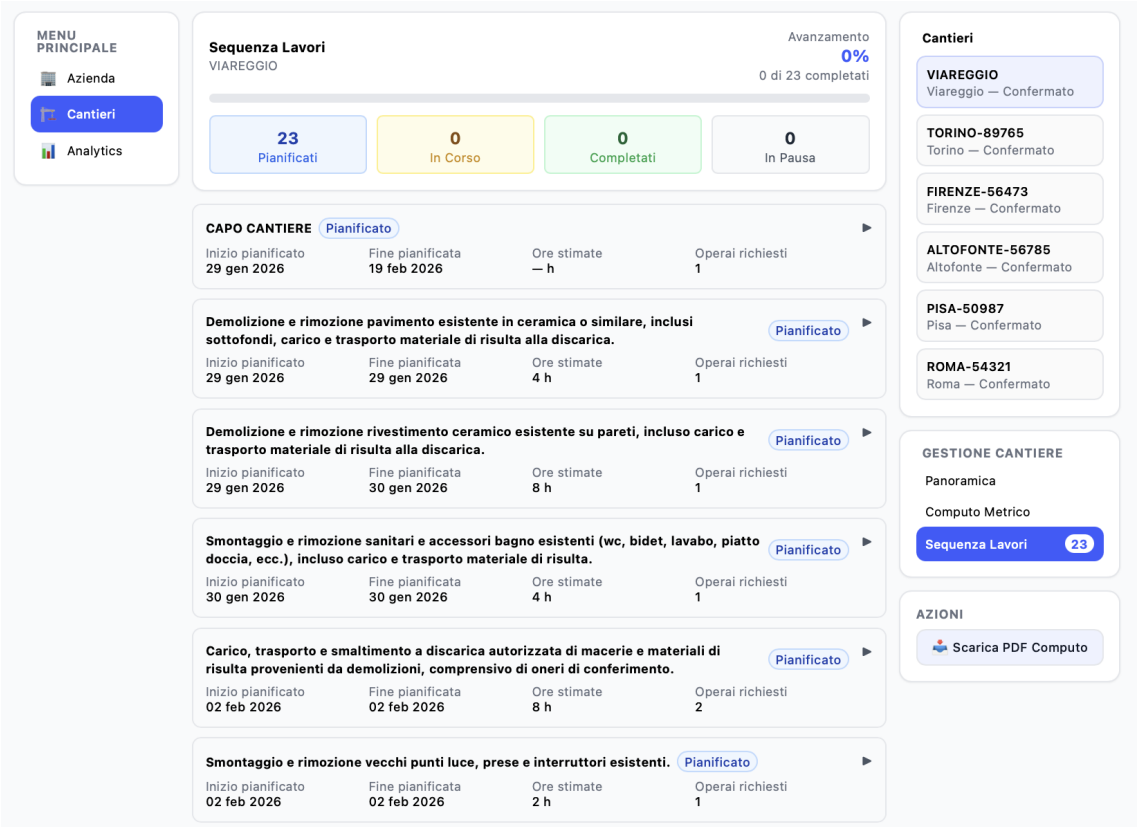

Figure 7.7: Work Sequence: tasks with assigned workers

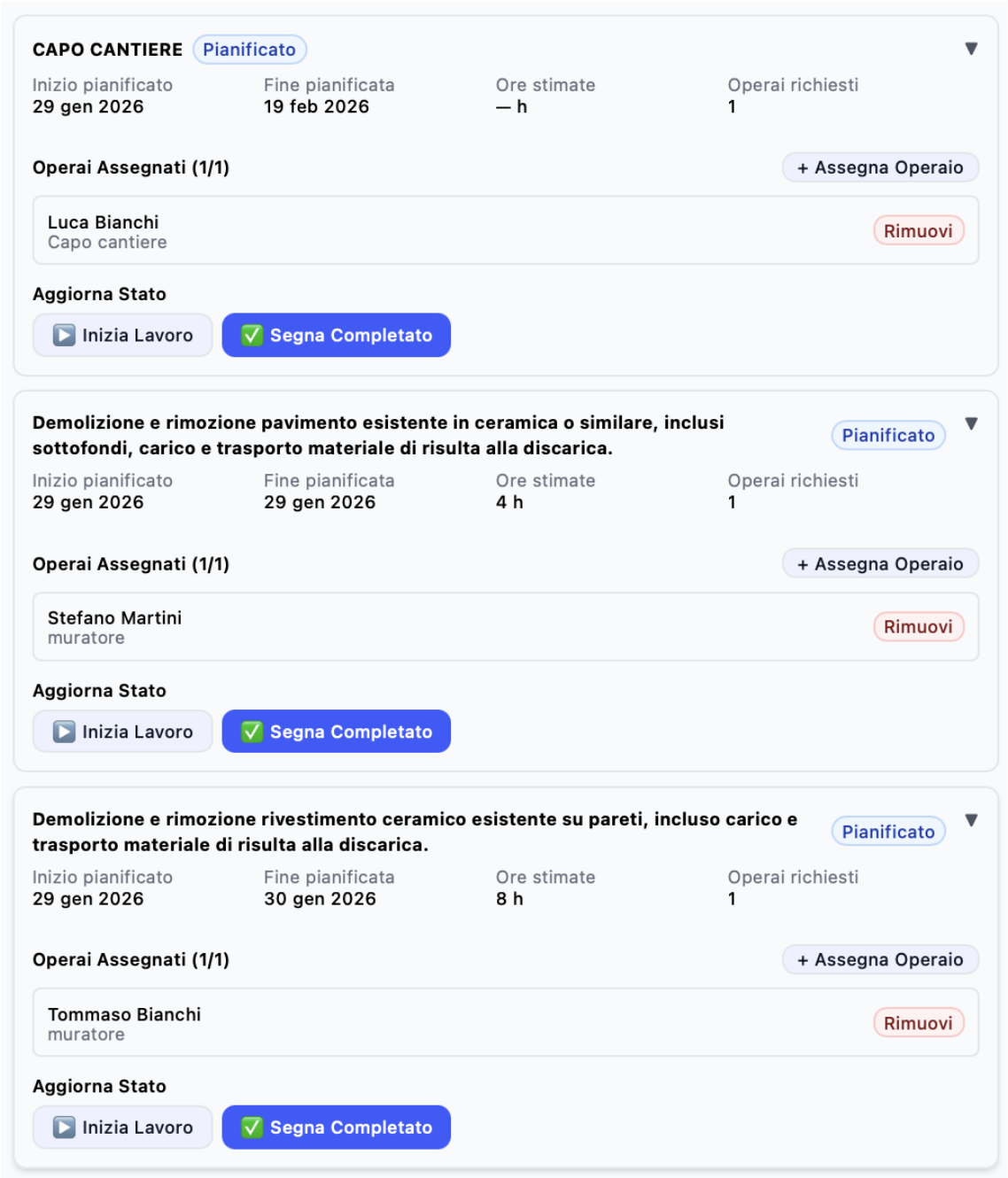

Figure 7.8: Task detail: assigned workers overview

7.3 Success Criteria

The demo is considered successful if:

- The generated bill of quantities reflects the content of the input file.
- No worker overlap conflicts (overbooking) are generated.
- The assistant correctly answers questions about Costs, Times, and Resources.